configuration file so you could control the availability dynamically at runtime, but for this example I'm using a hardcoded value.

TIP To read more about using feature toggles in your applications, see my series "Adding feature flags to an ASP.NET Core app" at http://mng.bz/2e40.

This piece of code is self-contained cross-cutting logic, which is somewhat orthogonal to the main intent of each action method—a perfect candidate for a filter. You want to execute the feature toggle early in the pipeline, before any other logic, so a resource filter makes sense.

TIP Technically, you could also use an authorization filter for this example, but I'm following my own advice of "Don't write your own authorization filters!"

The next listing shows an implementation of `FeatureEnabledAttribute`, which extracts the logic from the action methods and moves it into the filter. I've also exposed the `IsEnabled` field as a property on the filter.

Listing 22.4 The `FeatureEnabledAttribute` resource filter

```
public class FeatureEnabledAttribute : Attribute, IResourceFilter
{
    public bool IsEnabled { get; set; }                ❶
    public void OnResourceExecuting(                   ❷
        ResourceExecutingContext context)              ❷
    {
        if (!IsEnabled)                                ❸
        {                                              ❸
            context.Result = new BadRequestResult();   ❸
        }                                              ❸
    }
    public void OnResourceExecuted(                    ❹
```